AI 코딩 에이전트 실패 조기 예측

팀원	학번	담당
손의권	20223515	---
차승준	20223843	---

코딩 에이전트가 GitHub 이슈를 푸는 도중 앞 N스텝만 보고 최종 실패 여부를 예측한다. 실패할 실행을 일찍 끊으면 API 비용과 GPU 시간을 줄일 수 있다.

1. 문제와 동기

에이전트가 이슈 하나를 풀 때 수십~수백 스텝을 실행하고, 스텝마다 지금까지의 맥락 전체를 다시 모델에 넣는다. 사용할 데이터에서 성공률은 16.7%다. 비용 대부분이 결국 실패할 실행에 들어간다.

항목	내용
입력	궤적의 앞 N스텝 (에이전트 행동 + 환경 응답)
타깃	최종 성공 여부 (이진 분류)
사용자	에이전트 서비스 운영자. 예측값으로 조기 종료, 재시작, 다른 모델로 교체를 결정한다
핵심 질문	N을 얼마나 줄여도 예측이 되는가. 고정 스텝 상한보다 비용을 더 줄이는가

선행 연구. 2026년에 이 문제를 다룬 논문이 여러 편 나왔다.

연구	방법	결과
FailFast-RestartSmart (Wang et al., 2026)	0.6B 모니터, Qwen3.6-27B 궤적, SWE-bench Verified	오탐 5%에서 토큰 14.6~20.4% 절약
EarlyEval (Shi et al., 2026)	행동·텍스트 피처 + LightGBM, 궤적 약 7.8천 개	스텝 26% 절약
Automata from Agent Traces (Cho et al., 2026)	상태기계 피처 + GBM	SWE-agent 전체 궤적 AUROC 0.799, 진행률(%) 기준 조기 평가
Last Step Matters (2026)	모델 불확실성 기반	진행 50% 시점 평균 AUROC 0.60 미만
SWE-Gym verifier (Pan et al., 2024)	완료된 궤적을 LLM으로 채점	조기 예측이 아니라 사후 선택

차별점. 확인한 범위에서 아래 세 가지를 주요 평가로 다룬 연구는 없다.

1. **이슈 난이도와 궤적 신호 분리.** 이 데이터는 이슈 하나당 궤적이 평균 22개다. 같은 이슈 안에서 성공한 궤적과 실패한 궤적을 구분할 수 있는지 따로 측정한다. 뒤의 데이터 분석에서 보듯 이 분리를 하지 않으면 AUROC가 이슈 난이도만으로 부풀려진다.
2. **절대 스텝 기준.** 진행률(%)로 자르는 평가는 전체 길이를 알아야 하므로 실제 운영에서 쓸 수 없다. N=3, 5, 8, 10, 15, 20 같은 절대 스텝 시점에서 예측한다.
3. **고정 스텝 상한과 비교.** "30스텝 넘으면 끊기" 같은 단순 규칙이 이미 강한 기준선이다. 예측 모델이 이 규칙보다 나은지를 같은 성공 손실률에서 비교한다.

2. 데이터

소스	규모	용도
nebius/SWE-agent-trajectories (HF, CC-BY-4.0)	궤적 80,036개, 이슈 3,591개, 에이전트 스텝 약 212만 개, parquet 1.1GB	학습·검증·테스트
SWE-bench/experiments	리더보드 제출작별 궤적 + 인스턴스별 성공 여부	외부 테스트 (선택)

SWE-agent 프레임워크에서 Llama 계열 모델 3종이 생성했다. 스텝은 `ai` 메시지(추론 + 명령) 1개와 그 뒤 `user` 메시지(환경 관측) 1개로 정의한다.

컬럼

컬럼	뜻	사용
`instance_id`	저장소 + 이슈 번호	그룹 분할 기준
`model_name`	궤적 생성 모델	모델 간 일반화 테스트
`target`	성공 여부	라벨
`trajectory`	스텝별 추론·행동·관측 메시지	앞 N스텝만 입력
`exit_status`, `generated_patch`, `eval_logs`	종료 상태, 최종 패치, 채점 로그	제외 (라벨 누수)

직접 확인한 통계 (HF parquet 전체 조회)

항목	값
모델별 궤적 수 (성공률)	Llama-70B 74,792 (16.7%), 8B 4,053 (15.1%), 405B 1,191 (25.9%)
이슈당 궤적 수	중앙값 16, 최대 125
이슈 구성	한 번도 성공 못 한 이슈 2,753개 (76.7%). 성공·실패가 섞인 이슈 807개, 궤적 34,922개
스텝 수 중앙값	성공 12, 실패 19 (최대 408)
N스텝 시점에 진행 중인 궤적의 성공률	N=10: 59,708개 중 13.8%, N=20: 34,059개 중 6.4%
같은 이슈의 다른 궤적 성공률만으로 예측한 AUROC	0.945

마지막 줄의 값은 학습 없이 같은 이슈의 다른 궤적 결과만 본 것이다. 결과가 대부분 이슈 난이도로 정해지므로 궤적을 보지 않아도 AUROC가 높게 나온다. 그래서 평가에서 이슈 난이도 기준선과 이슈 내 AUROC를 따로 둔다.

3. 방법

시점 N마다 N스텝 넘게 진행 중인 궤적만 모아 앞 N스텝으로 실패 확률을 예측한다. N 이전에 끝난 궤적은 그 시점에서 끊을 대상이 아니므로 제외한다.

단계	모델	입력	목적
B0	고정 스텝 상한, 에러 횟수 규칙	스텝 수, 에러 메시지 수	운영 기준선

단계	모델	입력	목적
B1	TF-IDF + 로지스틱 회귀	이슈 본문만 (N=0)	이슈 난이도 기준선
M1	로지스틱 회귀 → XGBoost	행동 종류별 빈도, 에러 횟수, 같은 명령 반복 횟수, 열어 본 파일 수, 편집 횟수, 관측 길이 + B1 점수	해석 가능한 피처 모델
M2	LSTM	스텝별 행동 토큰 시퀀스 (`open`, `edit`, `search`, `python` 등) + 스텝 피처	행동 순서 정보
M3	소형 Transformer 인코더 fine-tune (예: ModernBERT-base)	앞 N스텝 원문, 관측은 스텝당 앞부분만 잘라 사용	텍스트 의미 정보

M1은 B1 점수를 넣은 버전과 뺀 버전을 모두 학습해서 궤적 피처가 이슈 난이도 위에 더하는 정보량을 잰다. 피처 중요도(SHAP)로 실패를 가장 잘 알려 주는 초반 신호를 찾는다.

누수 점검

- `exit_status`, `generated_patch`, `eval_logs`는 입력에서 뺀다.
- 에이전트가 앞 스텝에서 직접 실행한 테스트 출력은 운영 중에도 보이는 정보라 허용한다. 채점용 숨김 테스트 결과가 앞 스텝에 섞여 있는지는 직접 확인한다.
- 시스템 프롬프트는 모든 궤적에서 같으므로 제외한다.
- 정답 패치를 알아야 계산되는 값(정답 파일 열람 여부 등)은 피처로 쓰지 않는다.

4. 평가

분할

- `instance_id` 기준 그룹 분할. 같은 이슈의 궤적이 학습과 테스트에 동시에 들어가면 이슈 난이도를 외워서 맞힌다.
- 8B·405B가 시도한 이슈 578개(16.1%)를 테스트 이슈로 고정한다. 학습은 나머지 이슈 3,013개의 70B 궤적(62,460개)으로 하고, 하이퍼파라미터는 학습 이슈 안에서 5-fold GroupKFold로 고른다.
- 같은 테스트 이슈에서 70B 궤적(12,332개)은 같은 모델 테스트, 8B·405B 궤적(5,244개)은 다른 모델 일반화 테스트로 쓴다. 처음에는 모델 2종으로 학습하고 나머지 1종으로 테스트하려 했다. 그러나 70B가 전체 궤적의 93%라 70B를 테스트로 빼면 학습 데이터가 5,244개만 남고, 70B로 학습하고 8B·405B로 테스트하면 같은 이슈가 학습과 테스트에 모두 들어간다. 그래서 테스트 이슈를 먼저 고정하고, 그 안에서 모델별로 나눠 평가한다.
- 외부 테스트는 SWE-bench/experiments에서 SWE-agent 형식 제출작 2~3개를 골라 가능하면 진행한다.

지표

지표	설명
AUROC, PR-AUC	N별 곡선으로 보고. PR-AUC는 성공 클래스 기준으로 잰다. 성공은 소수(16.7%)이고 잘못 끊었을 때 손해가 가장 큰 쪽이라, 성공 궤적을 얼마나 놓치지 않는지 본다. 무작위 예측의 PR-AUC는 해당 시점의 성공률(N=10에서 0.138)과 같다
이슈 내 AUROC	테스트 이슈 중 성공·실패가 섞인 이슈 안에서만 계산 (전체 데이터 기준 807개). 이슈 난이도를 뺀 궤적 신호의 크기. 성공한 궤적은 대체로 일찍 끝나서 N이 커질수록 계산 가능한 이슈가 줄어들므로, N별로 계산에 쓴 이슈 수를 함께 보고
비용 곡선	성공 손실률(끊긴 성공 / 전체 성공) 대비 절약 스텝 비율. 맥락 길이를 반영한 절약 토큰 비율도 함께 보고
운영 비교	성공 손실 5% 이하에서 절약량을 고정 스텝 상한과 비교

고정 상한 기준선은 데이터에서 이미 계산했다.

상한 K	잃는 성공	절약 스텝
30	7.0%	29.4%
35	**4.9%**	**24.6%**
40	3.7%	20.8%
50	2.2%	15.8%

성공 손실 5% 이하를 만족하는 가장 작은 상한은 K=35이고, 이때 스텝 24.6%를 절약한다. 예측 모델은 성공 손실 5% 이하에서 24.6%보다 많이 절약해야 의미가 있다. 상한과 예측기를 함께 쓰는 정책도 비교한다.

5. 예상 결과

- N별 AUROC 곡선과 비용 곡선. 몇 스텝에서 끊어야 쓸 만한지에 대한 구체적인 값.
- 예측력 분해: 이슈 난이도(B1)만으로 맞히는 부분과 궤적 행동이 더하는 부분. 이슈 내 AUROC가 0.5에 가깝다면 "초반 궤적으로 실패를 안다"는 주장 대부분이 이슈 난이도 효과라는 결론이 되며, 이 경우에도 결과로 보고한다.
- 실패를 가장 잘 알려 주는 초반 신호. 후보는 같은 명령 반복, 에러 누적, 이슈 본문에 언급된 파일을 열지 않는 탐색이다. 데이터 카드에서 정답 파일을 한 번이라도 연 비율은 성공 83%, 실패 40%였다. 정답 파일은 정답 패치를 알아야 정해지므로 운영 중에는 계산할 수 없다. 이 값은 피처로 쓰지 않고 사후 분석에만 쓴다.
- 고전 ML, 시퀀스 모델, 텍스트 모델 중 어느 쪽이 N이 작을 때 유리한지.
- 운영자가 쓸 수 있는 정책 예시: "N=10 시점에 실패 확률이 τ 이상이면 종료"로 성공을 x% 잃고 비용을 y% 줄인다.

6. 리스크와 대응

리스크	대응
스텝 수 자체가 강한 신호라 모델이 "아직 안 끝났다"만 학습	시점별로 진행 중인 궤적만 비교하고 고정 상한 기준선과 대조
이슈 난이도가 AUROC를 부풀림	그룹 분할, B1 기준선, 이슈 내 AUROC
Llama 계열 + SWE-agent 한 프레임워크라 일반화가 불확실	8B·405B 테스트, 가능하면 외부 제출작 테스트. 결과의 적용 범위를 명시
Transformer 학습 비용 (궤적 8만 개 × 앞부분 1~2천 토큰, VRAM 12GB)	관측을 잘라 입력 길이 제한, 혼합 정밀도 사용. 시간이 부족하면 N 값을 줄여 학습
데이터 규모 (parquet 1.1GB)	처리 단위는 에이전트 스텝 약 212만 개의 텍스트. 파싱과 피처 추출을 시점 N마다 반복한다

7. 자원과 일정

Ryzen 9 9950X, RAM 64GB, RTX 5070 12GB. 파싱과 피처 추출은 CPU로 30분 이내, XGBoost는 실험당 수 분, LSTM은 실험당 10~30분, Transformer fine-tune은 전체 반나절~하루로 예상한다. 저장 공간은 10GB 미만.

주차	작업
8주차	궤적 파싱, 스텝 정의 확정, 누수 점검, 그룹 분할 확정, 기준선 B0·B1
9주차	M1 피처 추출, 로지스틱 회귀·XGBoost 학습, N별 AUROC·이슈 내 AUROC
10주차	비용 곡선, 고정 상한 대비 비교, SHAP 분석, M2(LSTM) 학습
11주차	M3(Transformer) fine-tune, 8B·405B 모델 간 일반화 테스트

주차	작업
12주차	외부 테스트(선택), 결과 정리, 발표 자료 작성

최종 발표는 13~14주차다. M3나 외부 테스트가 늦어지면 B0·B1·M1·M2 결과로 발표하고, M3는 가능한 N 값까지만 보고한다.

참고문헌

- Wang et al. *Fail-Fast, Restart-Smart: Early Failure Prediction and Restart for SWE Agentic Tasks.* arXiv:2608.03222, 2026.
- Shi et al. *EarlyEval: Cheaper Agent Evaluation via Early Outcome Prediction.* arXiv:2609.02783, 2026.
- Cho et al. *Automata from Agent Traces: Failure and Next-Step Prediction.* arXiv:2608.23670, 2026.
- *Last Step Matters: Early Uncertainty Cannot Predict Failure in Long-Horizon Agents.* arXiv:2608.29685, 2026.
- Pan et al. *Training Software Engineering Agents and Verifiers with SWE-Gym.* arXiv:2412.21139, 2024.
- Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* arXiv:2405.15793, 2024.
- Nebius. *SWE-agent-trajectories.* Hugging Face Datasets.